Sign Recognition Project Report: American Sign Language (ASL) Recognition

Simone Pellegatta (ID: 1127692), Sofia Aikaterini Michanetzi (ID: 1100945)

June 25, 2026

Abstract

This report describes the implementation of a real-time American Sign Language (ASL) recognition system. The system bridges the LabVIEW graphical interface with state-of-the-art Python-based artificial intelligence models (SigLIP), enabling visual gesture recognition and user interaction.

1 Introduction

This project addressed the problem of translating user gestures from sign language into text. For the experimental evaluation of the system, an interface was built in LabVIEW, through which the user can interact with the camera, select image sources, and observe the algorithm's predictions.

2 System Structure and Operation

2.1 Experimentation Capabilities

The system user can examine:

1. The recognition of American Sign Language letters in real-time via the Next_letter button.
2. The vocalization of the generated text via the Speak button.

Additionally, features such as the following are supported:

- Switching between a live camera and static photos (via the Input switch, which in the case of a static photo reads the Filename entered by the user).
- Concatenating letters or spaces (via the Space button) to form words.
- Switching Sign Language Recognition between right-handed and left-handed (via the Left Hand / Right Hand switch, which inputs the mirror image in the case of a left-handed user) for word formation.

2.2 Graphical Interface (Front Panel)

The user interface was designed in LabVIEW. Figure 1 shows the graphical interface where the user views the camera and the prediction result, while Figure 2 presents the corresponding Block Diagram.

2.3 Detailed Operational Description

The system operates via a client-server architecture that allows for real-time recognition of American Sign Language gestures.

Initially, the user provides input data either through a live camera stream or by selecting a static image. This input data is captured and processed by the graphical interface developed in LabVIEW, which is responsible for image acquisition and user interaction.

Once an image is acquired, it is sent via an HTTP request to a backend server implemented in Python. This server hosts the SigLIP deep learning model, which processes the image and extracts

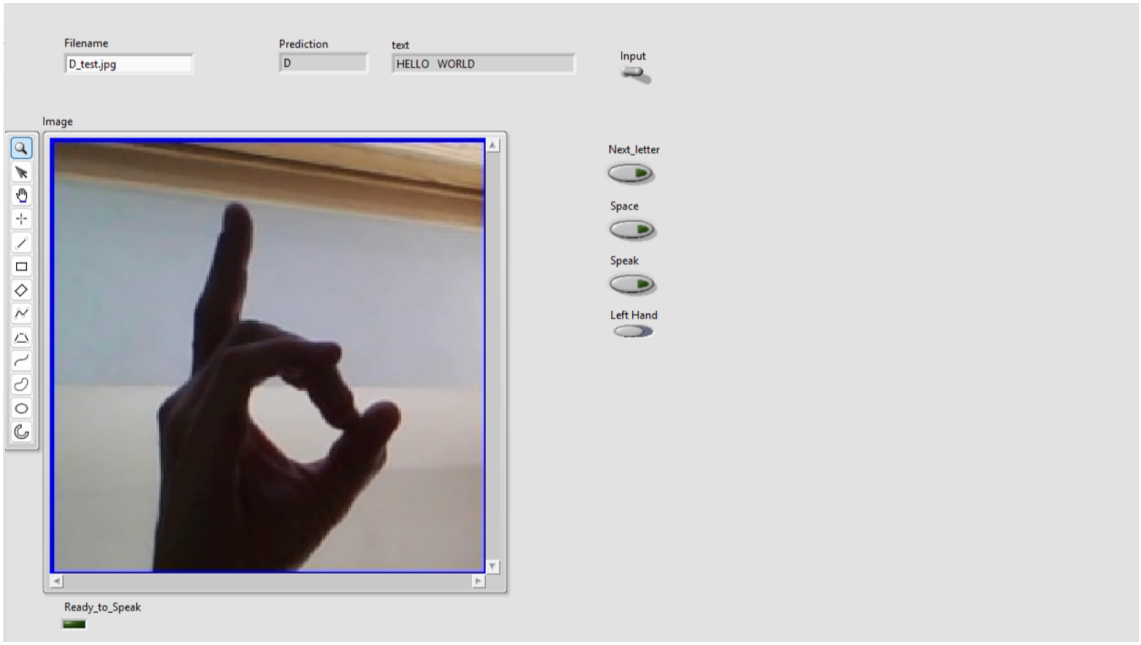

Figure 1: The system's Interface in LabVIEW, showing the image capture and the result indicator.

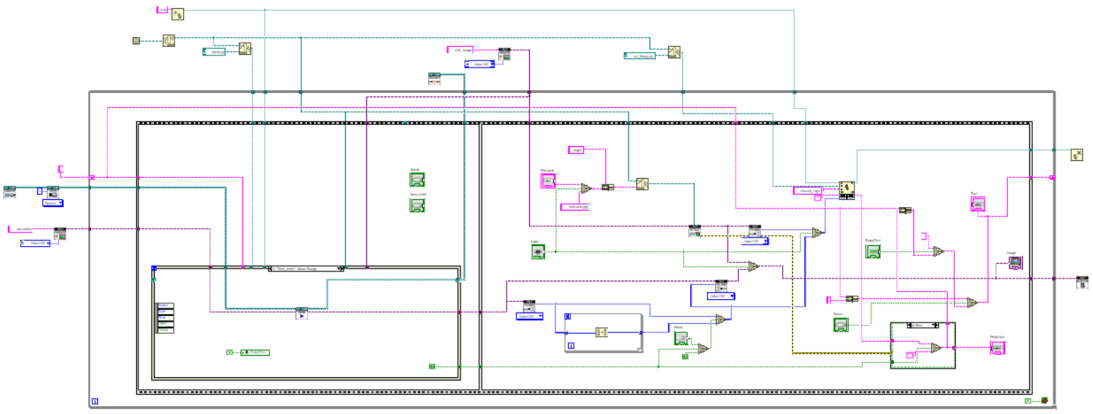

Figure 2: The system's Block Diagram in LabVIEW, showing the operation of the American Sign Language recognition system.

significant features. The model then calculates prediction scores for each possible letter and selects the most probable one.

The prediction is returned to the LabVIEW client via an HTTP response. The interface then displays the recognized letter in real-time, allowing the user to gradually form words using additional control elements like Next_letter and Space.

Finally, the system provides optional text-to-speech functionality, enabling the conversion of the generated text into audio output.

2.4 Hardware and Software

The system's architecture requires the separation of software layers, as shown in Table 1 below.

Layer	**Software**	**Architecture**
Graphical Interface	LabVIEW	32-bit
Intermediate Bridge	Python (Requests)	32-bit
AI Server	Python (Flask, PyTorch)	64-bit

Table 1: Software distribution and versions.

The separation of these layers and the use of different Python versions (32-bit and 64-bit) was the only way to successfully implement the system, due to the following technical reasons:

- **Library Compatibility:** LabVIEW's graphical interface and image acquisition (NI Vision) operate in a 32-bit environment, forcing the Python Node to exclusively call 32-bit Python. However, modern and heavy Deep Learning models (like SigLIP) and libraries like PyTorch no longer support 32-bit systems and strictly require a 64-bit environment.
- **Memory Limits:** A 32-bit application has a strict limit on RAM usage (approximately 2-4 GB), which is insufficient for loading a neural network. The 64-bit server frees the system from this constraint.
- **Process Decoupling:** Through the Client-Server architecture and the use of HTTP requests, the video stream and the responsiveness of the LabVIEW Interface remain smooth and unaffected by the heavy processing load of the AI.

2.5 Mathematical Background

Image classification is performed by the neural network (SigLIP). After the model processes the pixel array, it outputs a score vector $z_1, z_2, \ldots, z_K$ for each possible letter. The final probability p_i for letter i is calculated using the Softmax function:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

where $K = 26$ is the number of letters in the Latin alphabet. The server returns the letter with the highest probability, which is then registered in the system as the new prediction.

2.6 Conclusions

In conclusion, the implementation of this project successfully overcame the compatibility limitations of the different software, creating a functional and easy-to-use accessibility tool. The communication bridge worked flawlessly, delivering sign language prediction in real-time. Future improvements could include dynamic motion recognition, rather than just static letters, as well as expansion to other sign languages beyond ASL.

References

[1] National Instruments. *LabVIEW and NI Vision Development Module Documentation*. https://www.ni.com

[2] Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*.

[3] Hugging Face. *Alphabet-Sign-Language-Detection*. https://huggingface.co/prithivMLmods/Alphabet-Sign-Language-Detection

[4] *OpenLvVision Reference Documentation*. https://openlvvision.org/docs/